

# claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf\_1ee5ffdf-362\agent-a2bfa7c17bbccfbc2.jsonl`
- SHA-256: `d37a4c3302c9d0408c90f76a5486167434e5e4e71d159ad260d5ea92f8450938`
- Source modified: `2026-06-22T16:13:59+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_1ee5ffdf-362`
- session\_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`

## Transcript

### 1. user (2026-06-22T16:12:14.848Z)

Review the UNCOMMITTED M11 change in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel (run 'git -C "C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel" diff' + read the two new files backend/flywheel.py and tests/test\_flywheel.py).  
WHAT IT IS: a default-OFF, privacy-sensitive "data flywheel" skeleton that, when activated, would CAPTURE a user's uploaded video + rally output into a per-video workspace for later use as TRAINING data. It MUST be impossible for it to capture anything in production today.  
The design: backend/flywheel.py has consent\_attested() (a FAKED hard-deny returning False always), capture\_job(), shadow\_eval() (delegates to backend.eval.experiment.compare/compare\_unlabeled), propose\_promotion() (delegates to backend.eval.promotion.promote\_if\_served\_safe), and run\_for\_job() (the gated orchestrator).  
FlywheelConfig (enabled=False default) added to AppConfig. backend/api/job\_worker.py got a gated \_maybe\_run\_flywheel hook (mirrors the M8 \_maybe\_record\_job\_cost) called at job completion. No DB migration.  
Read for grounding: backend/flywheel.py, backend/api/job\_worker.py (the hook + the M8 cost hook it mirrors), backend/config/models.py (FlywheelConfig + AppConfig), backend/utils/workspace.py (VideoWorkspace.outputs/source are PROPERTIES; for\_video; resolve\_workspace\_root), backend/eval/experiment.py (compare/compare\_unlabeled signatures), backend/eval/promotion.py (promote\_if\_served\_safe ? confirm the A/B argument order). Be adversarial; default to a finding if uncertain.

LENS = PRIVACY / INERTNESS (the most important). Try to find ANY path where user data could be captured in production today, or where the guardrail could be accidentally weakened:

1. Trace run\_for\_job + \_maybe\_run\_flywheel: is capture TRULY gated behind BOTH flywheel.enabled (default False) AND consent\_attested (faked-deny)? Could either guard be bypassed (e.g. a code path that captures before the consent check, or a default that's truthy)?
2. consent\_attested returns False always ? confirm NOTHING in the diff calls capture\_job WITHOUT first passing consent\_attested. Is the faked-deny clearly documented as "do not relax without counsel"?
3. Does the job-completion hook risk capturing on EVERY job once someone flips flywheel.enabled, EVEN without real consent wiring? (It shouldn't ? consent\_attested still denies. Confirm.)
4. Does capture\_job copy the SOURCE VIDEO? Under what conditions? Is that gated? (It's only reached via run\_for\_job which is consent-gated, but verify capture\_job itself isn't called elsewhere unguarded.)
5. Any logging that could leak PII (owner\_id, paths, video content)?

Return inert\_guarantee + findings with file:line.

### 2. assistant (2026-06-22T16:12:16.833Z)

I'll analyze this M11 flywheel change for privacy and inertness vulnerabilities. Let me start by examining the diff and then reading the key files.

### 3. user (2026-06-22T16:12:17.502Z)

```

diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 50642ed..aae876c 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -89,6 +89,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
    db_instance.set_job_video_id(job_id, result["video_id"])
    db_instance.mark_job_done(job_id, result)
    _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
+   _maybe_run_flywheel(job, result)    # M11 data-flywheel (default-OFF + consent-faked-
deny)
    except _main.JobWaiting as w:
        # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
        logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -129,6 +130,28 @@ def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) ->
None:
    logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)

+def _maybe_run_flywheel(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+   """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a no-op
unless
+   ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-deny**
today
+   (``flywheel.consent_attested`` returns False for every job until counsel + the consent
schema
+   land), so this captures **nothing** in production. When activated it captures the consented
job's
+   artifacts into the per-video workspace (? later labeling ? the golden training set).
+
+   Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is the
claimed
+   row (carries ``owner_id``); ``result`` is the pipeline output."""
+   import backend.main as _main
+
+   flywheel_cfg = getattr(_main.global_config, "flywheel", None)
+   if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
+       return
+   try:
+       from backend import flywheel
+
+       flywheel.run_for_job(job, result, _main.global_config)
+   except Exception as e: # never wedge a completed job on the flywheel
+       logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
+
+   def _drain_due_jobs() -> int:
+       """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
`next_poll_at` is due), earliest-due first, until the table has nothing runnable.
diff --git a/backend/config/models.py b/backend/config/models.py
index d77f98e..9c8cf77 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -384,6 +384,26 @@ class BillingConfig(BaseModel):
    margin: float = 0.0
                                     # resale markup (?0); 0 = bill at cost

+class FlywheelConfig(BaseModel):
+   """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
+   loop: on a CONSENTED adult job, durably **capture** the source + rally output into the per-
video
+   workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden TRAINING
set, so
+   the owned model climbs toward the D11 ship floor.
+
+

```

```

+    **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-identical
to
+    today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
+    captures **nothing**, because ``flywheel.consent_attested`` currently returns ``False`` for
every
+    job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-delete-
raw).
+    This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
+    wiring) + the owned-model + golden-label data needed to *activate* shadow-eval/promotion
are
+    owner/counsel/GPU scope. Pairs with M9-consent."""
+
+    enabled: bool = False
+    workspace_prefix: str = "flywheel"      # per-video workspace prefix for captured artifacts
+    shadow_eval: bool = True                 # run the owned-vs-Gemini shadow comparison on capture
+    min_promote_n: int = 5                   # promote_if_served_safe `min_n` (the served-safe
gate)
+
+
+
class AppConfig(BaseModel):
    """Root configuration object.

@@ -406,6 +426,7 @@ class AppConfig(BaseModel):
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
    billing: BillingConfig = Field(default_factory=BillingConfig)
+    flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)

    # Allow extra keys during transition so existing config.json files do not break.
    model_config = {
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index ec9f84b..1576e75 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -123,7 +123,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
| M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds,
bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
| ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/288) (ledger+pricebook+cost ? default-OFF; real prices + usage-emission ?
owner/follow-up) |
| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+
exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root`
jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ?
owner/counsel) |
| M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
approval | ? | ? |
-| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | ? |
+| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | ? |
+| **skeleton landed** (see changelog):
`backend/flywheel.py` + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook,
**consent faked-deny** ; live activation ? owner/counsel (consent schema) + owned-model/golden
data |

```

| **M12** | **gdrive-api** (OAuth) StorageBackend (D14 Northstar): **cloud** reads the source from + writes the stitched reel **back** to the user's Google Drive next to the source (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

**Testing strategy is a first-class deliverable** ?

[`TESTING_STRATEGY.md`](`TESTING_STRATEGY.md`). **Run logs / sample runs for posterity** ?  
[`runlogs/`](`runlogs/`).

@@ -168,3 +168,4 @@ Legend: ? Todo . ? In progress . ? Done . ? Gated. **One small PR per**  
- 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via `execution_policy.poll_until` + the storage layer (D16); `get_compute_backend` returns a backend ONLY for `compute_target=cloud_run` (else `None` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced `local_gpu+native`, never re-dispatch); the worker writes a `--done-uri` completion marker on success.  
`deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **M6** ? ? the real build/push/L4 run + `DEPLOY_REPORT` is M7 (owner, D17 budget).  
- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer/cmd_train` ? rc 2), **default-OFF** (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **M5** ? ? the real-GPU truly-local runlog (L5) is owner-side (template pre-staged in `runlogs/`).  
- 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new `include_exposure=True` from `_enforce_startup_or_exit`; the worker stays unaffected ? it reads `--video-uri`, not untrusted tenant paths). Gated on `security.edge_auth_enabled` (**default-OFF** ? strict no-op, byte-identical to today): when the owner flips edge auth ON to expose the engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED_NO_VIDEO_ROOT` (**fatal**: `allowed_video_root` jail unset ? a tenant could read an arbitrary local file), `EXPOSED_EDGE_AUTH_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-request 401 to boot), `EXPOSED_AUTH_EXTRA_MISSING` (**warn**: `PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +15 unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-exit, WARN-only-no-exit). Adversarial 3-lens review (`wf_35b735a9`) confirmed default-OFF byte-identical + no bypass; folded a **whitespace-drift** (aligned `auth.resolve_tenant` to `.strip()` team/AUD so "configured" means the same at boot + at request time) + named the config fields in the edge-auth-incomplete message. Pairs with the appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy clean. **M9** ? the actual exposure flip + CF team/AUD/ingress + key rotation stay owner-side. (Deferred to the PR-2 runbook: document that `upload_dir` must sit under `allowed_video_root`, and that you MUST set `edge_auth_enabled=True` when exposing ? these checks only fire then).  
+- 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12): `backend/flywheel.py` (`capture_job` ? per-video `VideoWorkspace` via `backend.storage.put`; `shadow_eval` ? `experiment.compare/compare_unlabeled`; `propose_promotion` ? `promotion.promote_if_served_safe` with Gemini=control/owned=proposal; `run_for_job` orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `maybe_run_flywheel` job-completion hook mirroring M8's cost hook. **TWO** independent guards keep it INERT in production: `flywheel.enabled=False` (default) AND `consent_attested` is a **FAKED hard-deny** (returns False for every job) ? so even enabled it captures **nothing** until the real consent schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage code so it works the day consent + the owned model + golden labels are wired. **NO DB migration** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote, the hook is best-effort). Suite green, ruff+mypy clean. **M11** ? ? LIVE activation is owner/counsel (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model + golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v8` (v7

was taken by the `locale` column).\*\*

#### 4. assistant (2026-06-22T16:12:19.983Z)

Now let me read the new files and key related code:

#### 5. user (2026-06-22T16:12:20.951Z)

```
1      """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? capture ? shadow-eval ?
promote.
2
3      The consent?capture?eval?goldenize loop: on a **consented adult** job, durably
**capture** the source
4      + rally output into the per-video workspace ? **shadow-eval** the owned model vs Gemini
? **propose
5      promotion** of good ones into the golden TRAINING set, so the owned model climbs to the
D11 ship floor.
6
7      ? **DEFAULT-OFF + INERT ? two independent guards keep this from doing ANYTHING in
production today:**
8      1. ``config.flywheel.enabled`` defaults ``False`` ? the job-completion hook is a
strict no-op.
9      2. :func:`consent_attested` is a **FAKED hard-deny** (returns ``False`` for every job)
until the real
10     consent schema lands (counsel-reviewed ToS/DPA ? consent columns ? wiring). So even
11     ``enabled=True`` captures **nothing**. This is the privacy guardrail ? D10 + the
CLAUDE.md rule:
12     adults-only, explicit opt-in, train-on-derived + auto-delete-raw; **minors
excluded**; per-user
13     training is a *separate* opt-in; paid tier never trains.
14
15     This module is the PLUMBING ? it REUSES the real ``workspace`` / ``eval.experiment`` /
16     ``eval.promotion`` / ``storage`` code so the loop works the day consent + the owned
model + golden
17     labels are wired. It ships **no live behavior change**. Pairs with M9-consent (the
consent columns).
18     """
19     from __future__ import annotations
20
21     import json
22     import logging
23     import os
24     import tempfile
25     from typing import Any, Dict, Optional, Sequence
26
27     from backend import storage as _storage
28     from backend.utils import workspace as _workspace
29
30     logger = logging.getLogger(__name__)
31
32
33     def _sc(storage_config: Any) -> Optional[Dict[str, Any]]:
34         """Normalize an ``AppConfig.storage`` (pydantic model / dict / None) to the dict
35         ``backend.storage`` expects."""
36         if storage_config is None:
37             return None
38         if hasattr(storage_config, "model_dump"):
39             return storage_config.model_dump()
40         return storage_config if isinstance(storage_config, dict) else None
41
42
43     def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
44         """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).
45
46         The flywheel may capture a user's footage into the training pool ONLY with attested,
```

```

**adults-only**
47     consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
separate explicit
48     opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent
columns + an
49     adults-only attestation + a retention class ? lands with **M9-consent**; until then
this denies
50     every job so the harness captures nothing. **Do not relax without counsel sign-
off.**"""
51     return False
52
53
54     def capture_job(
55         workspace: "_workspace.VideoWorkspace",
56         *,
57         rally_output: Dict[str, Any],
58         source_path: Optional[str] = None,
59         storage_config: Any = None,
60     ) -> Dict[str, str]:
61         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
62         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
63         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
64         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``
65         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""
66         written: Dict[str, str] = {}
67         cfg = _sc(storage_config)
68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
fh, indent=2)
72                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
73                 written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
74                 if source_path and os.path.exists(source_path):
75                     source_uri = _workspace.join_uri(workspace.source,
os.path.basename(source_path))
76                     written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
77             finally:
78                 try:
79                     os.unlink(manifest_local)
80                 except OSError:
81                     pass
82             return written
83
84
85     def shadow_eval(
86         candidates: Sequence[Any],
87         owned_variant: Any,
88         gemini_variant: Any,
89         *,
90         labeled: bool = False,
91         iou: float = 0.5,
92     ) -> Dict[str, Any]:
93         """Owned-vs-Gemini shadow comparison (the D11 climb signal). Delegates to the
experiment harness:
94         a labeled golden corpus ? :func:`backend.eval.experiment.compare` (B?A deltas +
honest CIs); an
95         unlabeled single video ? :func:`backend.eval.experiment.compare_unlabeled`
(agreement). The owned
96         model + a golden corpus are owner/GPU-side, so this only *runs* once they exist; the

```

```

wrapper makes
97     the call site real today (and is exercised via tests with stub variants)."""
98     from backend.eval import experiment
99
100     if labeled:
101         return experiment.compare(candidates, owned_variant, gemini_variant, iou=iou)
102     if len(candidates) != 1:
103         raise ValueError("unlabeled shadow_eval compares exactly one video at a time")
104     return experiment.compare_unlabeled(candidates[0], owned_variant, gemini_variant,
agree_iou=iou)
105
106
107     def propose_promotion(
108         served_owned: Sequence[float],
109         served_gemini: Sequence[float],
110         *,
111         min_n: int = 5,
112     ) -> Any:
113         """Whether the owned model is **served-safe** to promote toward default ? wraps
114         :func:`backend.eval.promotion.promote_if_served_safe` (the foundational *no-
regression-at-served-
115         windowing* gate). ``served_`` are per-video served F1s on the same corpus; Gemini
is the control
116         (A), the owned model the proposal (B)."""
117         from backend.eval.promotion import promote_if_served_safe
118
119         return promote_if_served_safe(served_gemini, served_owned, min_n=min_n)
120
121
122     def run_for_job(
123         job: Dict[str, Any],
124         result: Dict[str, Any],
125         config: Any,
126         *,
127         source_path: Optional[str] = None,
128     ) -> Optional[Dict[str, Any]]:
129         """The gated orchestrator called at job completion. Returns ``None`` (a no-op)
unless **both**
130         ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it never
captures (consent is
131         faked-deny). On a consented job it captures the artifacts into the per-video
workspace; shadow-eval
132         + promotion are wired but stay inert until the owned model + golden labels exist.
Best-effort: the
133         caller wraps this so a bookkeeping hiccup never wedges a completed job."""
134         fw = getattr(config, "flywheel", None)
135         if not fw or not getattr(fw, "enabled", False):
136             return None
137         if not consent_attested(job, config):
138             logger.info("[FLYWHEEL] job %s not captured ? no attested consent (faked-
deny).", job.get("id"))
139             return None
140
141         video_id = (result or {}).get("video_id")
142         if not video_id:
143             logger.info("[FLYWHEEL] job %s has no video_id ? nothing to capture.",
job.get("id"))
144             return None
145
146         storage_cfg = getattr(config, "storage", None)
147         output_dir = getattr(getattr(config, "output", None), "output_dir", "output")
148         root = _workspace.resolve_workspace_root(
149             getattr(storage_cfg, "workspace_root", "") or "",
150             getattr(storage_cfg, "output_root", "") or "",
151             output_dir,

```

```

152         )
153         ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
154         written = capture_job(
155             ws,
156             rally_output={"play_segments": (result or {}).get("play_segments", [])},
157             source_path=source_path,
158             storage_config=storage_cfg,
159         )
160         logger.info("[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id,
written)
161         return {"captured": written, "video_id": video_id}
162

```

## 6. user (2026-06-22T16:12:21.589Z)

```

1     """M11 data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12).
2
3     The cardinal invariants: **DEFAULT-OFF** (`flywheel.enabled=False` ? strict no-op) and
the
4     **faked consent hard-deny** (`consent_attested` returns False for every job ? nothing
is captured
5     even when enabled). Plus: the capture/shadow-eval/promote wrappers REUSE the real
workspace / eval /
6     promotion code (delegation verified). Pure + offline ? storage is the local (copy)
backend on tmp_path.
7     """
8     import json
9     import os
10
11     import pytest
12
13     from backend import flywheel
14     from backend.config import AppConfig
15
16
17     def _cfg(*, enabled=False, output_dir="output", prefix="flywheel"):
18         return AppConfig.model_validate({
19             "flywheel": {"enabled": enabled, "workspace_prefix": prefix},
20             "output": {"output_dir": output_dir},
21             "storage": {"backend": "local"},
22         })
23
24
25     # ----- default-OFF + consent
guardrail
26
27     def test_flywheel_config_defaults_off():
28         cfg = AppConfig()
29         assert cfg.flywheel.enabled is False
30         assert cfg.flywheel.workspace_prefix == "flywheel"
31
32
33     def test_consent_is_faked_hard_deny():
34         """The privacy guardrail: consent_attested denies EVERY job until counsel + the
schema land."""
35         assert flywheel.consent_attested({"id": 1}, _cfg(enabled=True)) is False
36         assert flywheel.consent_attested({"id": 2, "owner_id": "u@x"}, None) is False
37
38
39     def test_run_for_job_noop_when_disabled():
40         cfg = _cfg(enabled=False)
41         assert flywheel.run_for_job({"id": 1}, {"video_id": "v"}, cfg) is None
42
43
44     def test_run_for_job_noop_when_enabled_but_no_consent(tmp_path):

```

```

45         """enabled=True but consent denied (the faked default) ? still a no-op, NOTHING
written."""
46         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
47         assert flywheel.run_for_job({"id": 1}, {"video_id": "v"}, cfg) is None
48         assert not list(tmp_path.rglob("capture.json"))
49
50
51     # ----- capture
52     (consented)
53     def test_run_for_job_captures_when_enabled_and_consented(tmp_path, monkeypatch):
54         """With consent monkeypatched True, a per-video manifest lands under
<output>/flywheel/<vid>/."""
55         monkeypatch.setattr(flywheel, "consent_attested", lambda job, config=None: True)
56         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
57         out = flywheel.run_for_job(
58             {"id": 7}, {"video_id": "abc123", "play_segments": [{"id": 1, "start_time":
0}]}, cfg)
59         assert out and out["video_id"] == "abc123"
60         manifest = tmp_path / "flywheel" / "abc123" / "outputs" / "capture.json"
61         assert manifest.exists()
62         data = json.loads(manifest.read_text())
63         assert data["video_id"] == "abc123"
64         assert data["rally_output"]["play_segments"][0]["id"] == 1
65
66
67     def test_run_for_job_no_video_id_is_noop(tmp_path, monkeypatch):
68         monkeypatch.setattr(flywheel, "consent_attested", lambda job, config=None: True)
69         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
70         assert flywheel.run_for_job({"id": 7}, {}, cfg) is None
71
72
73     def test_capture_job_persists_manifest_and_source(tmp_path):
74         """capture_job writes the manifest AND (when given) the source video, via
storage.put."""
75         ws = flywheel._workspace.for_video(str(tmp_path), "vid9", prefix="flywheel")
76         src = tmp_path / "in.mp4"
77         src.write_bytes(b"FAKE-MP4")
78         written = flywheel.capture_job(
79             ws, rally_output={"play_segments": []}, source_path=str(src),
80             storage_config=AppConfig().storage)
81         assert os.path.exists(written["manifest_uri"])
82         assert os.path.exists(written["source_uri"])
83         assert (tmp_path / "flywheel" / "vid9" / "source" / "in.mp4").read_bytes() ==
b"FAKE-MP4"
84
85
86     # ----- shadow_eval / promote
87     delegation
88     def test_shadow_eval_labeled_delegates_to_compare(monkeypatch):
89         calls = {}
90         from backend.eval import experiment
91
92         def fake_compare(c, a, b, iou=0.5):
93             calls["compare"] = (c, a, b, iou)
94             return {"ok": 1}
95
96         monkeypatch.setattr(experiment, "compare", fake_compare)
97         res = flywheel.shadow_eval(["c1", "c2"], "owned", "gemini", labeled=True, iou=0.4)
98         assert res == {"ok": 1}
99         assert calls["compare"] == ([ "c1", "c2"], "owned", "gemini", 0.4)
100
101
102     def test_shadow_eval_unlabeled_delegates_to_compare_unlabeled(monkeypatch):

```

```

103     from backend.eval import experiment
104     seen = {}
105
106     def fake_unlabeled(v, a, b, agree_iou=0.5):
107         seen["u"] = (v, a, b, agree_iou)
108         return {"agree": 1}
109
110     monkeypatch.setattr(experiment, "compare_unlabeled", fake_unlabeled)
111     assert flywheel.shadow_eval(["only", "owned", "gemini"]) == {"agree": 1}
112     assert seen["u"] == ("only", "owned", "gemini", 0.5)
113
114
115     def test_shadow_eval_unlabeled_requires_single_video():
116         with pytest.raises(ValueError):
117             flywheel.shadow_eval(["a", "b"], "owned", "gemini", labeled=False)
118
119
120     def test_propose_promotion_delegates_with_gemini_as_control(monkeypatch):
121         """The owned model is the PROPOSAL (B); Gemini is the CONTROL (A) ? promote only if
served-safe."""
122         import backend.eval.promotion as promotion
123         seen = {}
124
125         def fake_gate(a, b, *, min_n=5):
126             seen["p"] = (list(a), list(b), min_n)
127             return "VERDICT"
128
129         monkeypatch.setattr(promotion, "promote_if_served_safe", fake_gate)
130         out = flywheel.propose_promotion([0.8, 0.9], [0.7, 0.75], min_n=6) # owned, gemini
131         assert out == "VERDICT"
132         assert seen["p"] == ([0.7, 0.75], [0.8, 0.9], 6) # A=gemini(control),
B=owned(proposal)
133
134
135     # ----- the job-worker
hook (gated)
136
137     def test_job_worker_hook_noop_when_disabled(monkeypatch):
138         """_maybe_run_flywheel must not call into the harness when flywheel is disabled."""
139         import backend.main as bmain
140         from backend.api import job_worker
141
142         monkeypatch.setattr(bmain, "global_config", _cfg(enabled=False), raising=False)
143         called = []
144         monkeypatch.setattr(flywheel, "run_for_job", lambda *a, **k: called.append(1))
145         job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"})
146         assert called == []
147
148
149     def test_job_worker_hook_is_best_effort(monkeypatch):
150         """An exception inside the flywheel never propagates (a completed job must not be
wedged)."""
151         import backend.main as bmain
152         from backend.api import job_worker
153
154         monkeypatch.setattr(bmain, "global_config", _cfg(enabled=True), raising=False)
155
156         def _boom(*a, **k):
157             raise RuntimeError("kaboom")
158
159         monkeypatch.setattr(flywheel, "run_for_job", _boom)
160         job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"}) # must NOT raise
161

```

```

1      """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3      See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4      ONE folder keyed by the MD5 ``video_id``
5      (``backend/utils/hashing.py::compute_video_id``):
6
7          <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9      The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
10     ``gdrive://``), so a
11     LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
12     the root
13     differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
14     and works
15     for every backend; the storage layer (``backend/storage/``) does the actual read/write.
16
17     Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
18     worth
19     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
20     extracted
21     chunks that must never be written to a cloud root. ``tmp/`` is the local-only bucket.
22
23     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
24     converges
25     in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
26     import.
27
28     """
29     from __future__ import annotations
30
31     import os
32     from dataclasses import dataclass
33
34     # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
35     compile
36     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
37     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
38     LOCAL_ONLY_SUBDIRS = ("tmp",)
39
40     def join_uri(root: str, *parts: str) -> str:
41         """Join path components onto a storage root with forward slashes.
42
43         Matches how the codebase already composes storage URIs (e.g.
44         ``f"{prefix}/weights/..."`` in
45         ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
46         and bare
47         paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
48         ``join_uri(root,
49         "x")``. Forward slashes are valid path separators in Python on Windows too.
50
51         """
52         base = root.rstrip("/")
53         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
54         inside a
55         # name (e.g. a mounted "My Drive" path) are preserved.
56         tail = "/" .join(p.strip("/") for p in parts if p and p.strip())
57         return f"{base}/{tail}" if tail else base
58
59     def resolve_workspace_root(workspace_root: str, output_root: str, output_dir: str) ->
60     str:
61         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
62         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
63         breaking)."""
64         return (workspace_root or "").strip() or (output_root or "").strip() or (output_dir
65         or "output")

```

```

50
51
52     @dataclass(frozen=True)
53     class VideoWorkspace:
54         """The single folder for one video. Pure path math; build with :func:`for_video`.
55
56         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
under it.
57         """
58
59         root: str
60         video_id: str
61         prefix: str = "workspaces"
62
63         @property
64         def base(self) -> str:
65             if self.prefix:
66                 return join_uri(self.root, self.prefix, self.video_id)
67                 return join_uri(self.root, self.video_id)
68
69         def path(self, sub: str = "", name: str = "") -> str:
70             """A path under this video's folder: ``base[/sub][/name]``."""
71             return join_uri(self.base, sub, name)
72
73         @property
74         def source(self) -> str:
75             return self.path("source")
76
77         @property
78         def detector(self) -> str:
79             return self.path("detector")
80
81         @property
82         def outputs(self) -> str:
83             return self.path("outputs")
84
85         @property
86         def reels(self) -> str:
87             return self.path("reels")
88
89         @property
90         def logs(self) -> str:
91             return self.path("logs")
92
93         @property
94         def tmp(self) -> str:
95             return self.path("tmp")
96
97
98     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> VideoWorkspace:
99         """Build the :class:`VideoWorkspace` for ``video_id`` under ``root`` (a storage
URI)."""
100         if not video_id:
101             raise ValueError("video_id is required to build a per-video workspace")
102         return VideoWorkspace(root=root, video_id=video_id, prefix=prefix)
103
104
105     def local_scratch_base(scratch_root: str, video_id: str) -> str:
106         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
cloud root).
107
108         Used for frame caches / extracted chunks (the ``tmp`` bucket) during compute,
before durable
109         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
since this

```

```

110         is always a local filesystem path.
111         """
112         if not video_id:
113             raise ValueError("video_id is required for a local scratch path")
114         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")
115

```

## 8. user (2026-06-22T16:12:23.585Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A Variant is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import glob
41    import json
42    import logging
43    import os
44    from dataclasses import dataclass, field
45    from datetime import datetime, timezone
46    from hashlib import sha1
47    from typing import Any, Callable, Dict, List, Optional, Sequence
48
49    from backend.eval.calibration import pct_improvement
50    from backend.eval.classifier import LogisticRegression
51    from backend.eval.gemini_refine import merge_overlaps
52    from backend.eval.metrics import Interval, match_intervals, score
53    from backend.eval.regression import gt_hash
54    from backend.eval.training import label_candidates
55    from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
56    from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count

```

```

ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt    # C2: calibrate cue
scores
58
59     logger = logging.getLogger(__name__)
60
61     # Where a comparison run appends one reproducible record. Tracked location (NOT under
62     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
63     # regression.DEFAULT_BASELINE_PATH.
64     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
65     _METRICS = ("precision", "recall", "f1", "mean_iou")
66
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73
74     # ----- Variant
75
76
77     @dataclass
78     class Variant:
79         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
80
81         A variant turns one video's persisted candidate windows + features into rally
82         intervals. Every knob defaults to the control behaviour (no gate, no learned
83         filter), so a variant that merely carries a new, disabled cue is byte-identical
84         to control ? the core no-regression guarantee.
85
86         Fields:
87         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
88           for the leaderboard and for selecting the served path (the existing
89           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
90           New cues are recorded here default-OFF.
91         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
92           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
93           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
94           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
95         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
96           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
97           cue persists that feature).
98         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
99           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
100           required for ``compare_unlabeled`` on a learned variant.
101         - ``feature_names`` ? the persisted features the learned filter consumes. When set
102           WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
103           (honest) ? the recipe, not a pinned artifact.
104         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
105           overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
106         """
107
108     name: str
109     segmenter: str = "wasb_hybrid"
110     config_overrides: Dict[str, Any] = field(default_factory=dict)
111     cue_flags: Dict[str, bool] = field(default_factory=dict)
112     min_crossings: int = 0
113     min_players: int = 0
114     classifier_model: Optional[str] = None
115     feature_names: Optional[List[str]] = None
116     threshold: float = 0.5
117     merge_gap: float = 1.0
118     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the

```

```

operating
119         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
120         # failure where a fixed 0.5 zeroed out a small-candidate video.
121         tune_threshold: bool = False           # pick the F1-best threshold on the train fold
122         calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
123
124         def is_learned(self) -> bool:
125             """True if this variant applies a learned classifier (pinned model or
recipe)."""
126             return self.classifier_model is not None or bool(self.feature_names)
127
128         def to_dict(self) -> dict:
129             return {
130                 "name": self.name,
131                 "segmenter": self.segmenter,
132                 "config_overrides": self.config_overrides,
133                 "cue_flags": self.cue_flags,
134                 "min_crossings": self.min_crossings,
135                 "min_players": self.min_players,
136                 "classifier_model": self.classifier_model,
137                 "feature_names": self.feature_names,
138                 "threshold": self.threshold,
139                 "merge_gap": self.merge_gap,
140                 "tune_threshold": self.tune_threshold,
141                 "calibrate": self.calibrate,
142             }
143
144         @classmethod
145         def from_dict(cls, d: dict) -> "Variant":
146             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
147             return cls(**{k: v for k, v in d.items() if k in known})
148
149         def spec_hash(self) -> str:
150             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
151             and makes a result reproducible. The pinned **control** variant always hashes
the
152             same, so a regression is always traceable to a recipe change, never to drift."""
153             blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154             return sha1(blob.encode()).hexdigest()[:12]
155
156
157         #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
158         #: filter. Always the comparison reference; "rollback" = make this the served variant.
159         CONTROL = Variant(name="control")
160
161
162         # ----- persisted candidates
163
164
165         @dataclass
166         class VideoCandidates:
167             """One video's persisted detection, ready for offline re-scoring.
168
169             ``intervals`` are the candidate windows; ``features`` is column-major
170             (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
171             ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
172             with ``load_video_candidates``, or directly in tests.
173             """
174
175             name: str
176             intervals: List[Interval]
177             features: Dict[str, List[float]] = field(default_factory=dict)
178             gts: Optional[List[Interval]] = None

```

```

179
180
181     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
182         """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
183         read (`shuttle` + `person`; `audio` stays optional ? it is added later by
184         `fusion_audio.augment` and older JSONs predate it).
185
186         The velocity-windowing / distill corpus-growth path persists a LEANER
187         `*_golden_features.json` row (`start`/`end` + velocity stats, no cue dicts)
for clips
188         that never went through fusion feature extraction. The loaders that build cue-rich
189         `VideoCandidates` from these files ? `ablation.load_videos`,
`fusion_compare.load_videos`
190         and `fusion_audio.load_videos_with_audio` ? SKIP those rather than crash on
`c["shuttle"]`,
191         so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
192         An empty candidate list is not a fusion file either.
193
194         Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
195         each landed an independent copy)."""
196         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
197
198
199     def load_golden_videos(
200         features_dir: str,
201         *,
202         shuttle_columns: Sequence[str],
203         person_columns: Sequence[str],
204         audio_columns: Sequence[str] = (),
205         audio_policy: str = "omit",
206         skip_logger: logging.Logger = logger,
207         skip_message: str,
208         no_audio_error: str = "{path} has no audio features",
209     ) -> List["VideoCandidates"]:
210         """Shared golden-feature corpus loader behind the three structurally-identical
loaders
211         (`ablation.load_videos` / `fusion_compare.load_videos` /
`fusion_audio.load_videos_with_audio`).
212
213         Reads every FULL-FUSION-SCHEMA `*_golden_features.json` in `features_dir` into a
214         `VideoCandidates` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215         velocity-windowing-only files (no `shuttle`/`person` cue dicts ? the schema the
216         corpus-growth path persists) are SKIPPED, not crashed on (see
`_is_fusion_schema`), so a
217         MIXED corpus loads the fusion subset cleanly.
218
219         The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
220         three callers differ ONLY in:
221
222         - `audio_policy` ? how the audio block is handled, checked BEFORE reading any
audio column
223         so a windowing-only file is skipped before either branch applies:
224         * `"omit"` ? no audio columns are loaded (`audio_columns`
ignored);
225         * `"tolerate-missing"` ? audio columns load, a missing block defaults them
to 0.0;
226         * `"require"` ? a fusion file lacking audio raises `SystemExit`
(via
227         `no_audio_error` formatted with `path`), mirroring "run `--augment`

```

```

first".
228         - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
229         emitted when a windowing-only file is skipped (kept byte-identical per caller).
230         """
231         if audio_policy not in ("omit", "tolerate-missing", "require"):
232             raise ValueError(
233                 f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}")
234         videos: List[VideoCandidates] = []
235         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
236             with open(path, encoding="utf-8") as f:
237                 d = json.load(f)
238                 cand = d["candidates"]
239                 if not _is_fusion_schema(cand):
240                     skip_logger.info(skip_message, os.path.basename(path))
241                     continue
242                 if audio_policy == "require" and "audio" not in cand[0]:
243                     raise SystemExit(no_audio_error.format(path=path))
244                 intervals = [(float(c["start"]), float(c["end"])) for c in cand]
245                 features: Dict[str, List[float]] = {}
246                 for fn in shuttle_columns:
247                     features[fn] = [float(c["shuttle"][fn]) for c in cand]
248                 for fn in person_columns:
249                     features[fn] = [float(c["person"][fn]) for c in cand]
250                 if audio_policy == "tolerate-missing":
251                     for fn in audio_columns:
252                         features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in
cand]
253                 elif audio_policy == "require":
254                     for fn in audio_columns:
255                         features[fn] = [float(c["audio"][fn]) for c in cand]
256                 gts = [(float(a), float(b)) for a, b in d["gts"]]
257                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
258                                         features=features, gts=gts))
259         return videos
260
261
262     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
263         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
264         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
265         col = vc.features.get(name)
266         n = len(vc.intervals)
267         if col is None:
268             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
269                             name, vc.name)
270             return [0.0] * n
271         if len(col) != n:
272             raise ExperimentError(
273                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
274         return [float(x) for x in col]
275
276
277     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
278         cols = [_feature_column(vc, name) for name in feature_names]
279         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
280
281
282     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
283         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
284         players)."""
285         n = len(vc.intervals)
286         mask = [True] * n
287         if variant.min_crossings > 0:

```

```

287         col = _feature_column(vc, "net_crossings")
288         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
289     if variant.min_players > 0:
290         col = _feature_column(vc, "players_in_court")
291         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
292     return mask
293
294
295     def predict(variant: Variant, vc: VideoCandidates,
296                 model: Optional[LogisticRegression] = None,
297                 threshold: Optional[float] = None,
298                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
299         """Variant prediction: gate by persisted features, optionally filter by a learned
300         model, then merge. Pure and deterministic.
301
302         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
303         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
304         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
305         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
306         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
307         them.
308
309         """
310         mask = _gate_mask(variant, vc)
311         if variant.feature_names:
312             if model is None:
313                 if variant.classifier_model is None:
314                     raise ExperimentError(
315                         f"variant {variant.name!r} is learned but has no model to predict "
316                         f"with (pass a fitted model or set classifier_model)")
317                 model = LogisticRegression.load(variant.classifier_model)
318             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
319 variant.feature_names))]
320             if calibrator is not None:
321                 proba = [calibrator(p) for p in proba]
322             thr = variant.threshold if threshold is None else threshold
323             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
324             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
325             return merge_overlaps(kept, gap_tol=variant.merge_gap)
326
327
328     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
329                             train_videos: Sequence[VideoCandidates], iou: float
330                             ) -> "tuple[Optional[Callable[[float], float]],
331 Optional[float]]":
332         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
333         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
334         (use the variant default). Honest: never looks at the held-out video.
335
336         """
337         calibrator: Optional[Callable[[float], float]] = None
338         if variant.calibrate:
339             raw: List[float] = []
340             y: List[int] = []
341             for vc in train_videos:
342                 raw.extend(float(p) for p in
343                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
344 [])))
345                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
346             if variant.calibrate == "platt":
347                 calibrator = fit_platt(raw, y)
348             elif variant.calibrate == "isotonic":
349                 calibrator = fit_isotonic(raw, y)
350             else:
351                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
352 'platt'/'isotonic')")
353             threshold: Optional[float] = None

```

```

347         if variant.tune_threshold:
348             best_t, best_f1 = variant.threshold, -1.0
349             for k in range(1, 20):                                     # grid 0.05..0.95 on the train
fold's rally F1
350                 t = round(0.05 * k, 2)
351                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
352                             vc.gts or [], iou).f1 for vc in train_videos]
353                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
354                 if mean_f1 > best_f1:
355                     best_f1, best_t = mean_f1, t
356                 threshold = best_t
357             return calibrator, threshold
358
359
360 # ----- variant scoring
361
362
363 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
364     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
365     evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
366     X: List[List[float]] = []
367     y: List[int] = []
368     for vc in videos:
369         if vc.gts is None:
370             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
371         X.extend(_feature_matrix(vc, variant.feature_names or []))
372         y.extend(label_candidates(vc.intervals, vc.gts, iou))
373     if not X:
374         raise ExperimentError("cannot train a learned variant on zero candidates")
375     return LogisticRegression().fit(X, y, variant.feature_names)
376
377
378 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
379                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
380     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
381     mean aggregate.
382
383     Prediction policy:
384     - **static variant** (no learned filter): predict each video directly ? no
training,
385     so no leakage; per-video scoring is already honest.
386     - **learned, pinned model** (`classifier_model` set): score every video with the
SHIPPED model. ? optimistic if those videos were in its training set ? use this
387     to
388     report "how the shipped artifact does here", not for the promotion decision.
389     - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
390     train on the OTHER videos, predict the held-out one. The number to trust.
391     - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
392     """
393     for vc in videos:
394         if vc.gts is None:
395             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
396
397     per_video: List[Dict[str, Any]] = []
398     predictions: Dict[str, List[Interval]] = {}
399
400     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
401     pinned_model = (LogisticRegression.load(variant.classifier_model)
402                     if variant.classifier_model is not None else None)

```

```

403     all_model = None
404     if recipe_lovo and not honest:
405         all_model = _train(variant, videos, iou)
406
407     for i, vc in enumerate(videos):
408         cal: Optional[Callable[[float], float]] = None
409         thr: Optional[float] = None
410         if recipe_lovo and honest:
411             others = [v for j, v in enumerate(videos) if j != i]
412             if not others:
413                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
414             model: Optional[LogisticRegression] = _train(variant, others, iou)
415             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
416                 assert model is not None # _train never returns
None
417                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
418             elif recipe_lovo: # honest=False ? one model over all
videos
419                 model = all_model
420             else: # static variant or pinned model
421                 model = pinned_model
422             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
423             assert vc.gts is not None # guarded at function top
424             sc = score(preds, vc.gts, iou)
425             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
426             predictions[vc.name] = preds
427
428             aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
429                         for m in _METRICS}
430         return {
431             "variant": variant.name,
432             "spec_hash": variant.spec_hash(),
433             "is_learned": variant.is_learned(),
434             "honest_lovo": recipe_lovo and honest,
435             "per_video": per_video,
436             "aggregate": aggregate,
437             "predictions": predictions,
438         }
439
440
441     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
442         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
443         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
444
445
446     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
447         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
448         gts_by_name = {v.name: (v.gts or []) for v in videos}
449         overlaps = _segm.DEFAULT_OVERLAPS
450         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
451         ratios: List[float] = []
452         for name, preds in eval_v.get("predictions", {}).items():
453             gts = gts_by_name.get(name, [])
454             got = _segm.f1_at_overlaps(preds, gts, overlaps)
455             for ov in overlaps:
456                 f1k[ov].append(got[ov])
457             ratios.append(_segm.segment_count_ratio(preds, gts))

```

```

458
459     def _mean(xs: List[float]) -> float:
460         return sum(xs) / len(xs) if xs else 0.0
461     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
462     out["segment_count_ratio"] = _mean(ratios)
463     return out
464
465
466     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
467                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
468         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
469
470         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
471         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
472         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
473         """
474         a_f1 = [p["f1"] for p in eval_a["per_video"]]
475         b_f1 = [p["f1"] for p in eval_b["per_video"]]
476         return {
477             "variant_a_ci": _ci_block(eval_a),
478             "variant_b_ci": _ci_block(eval_b),
479             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
480             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
481                             "variant_b": _segmentation_block(videos, eval_b)},
482         }
483
484
485     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
486               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
487         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
488         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
489
490         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
491         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
492         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
493
494         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
495         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
496         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
497         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
498         bare LOVO mean at n?6 is not trustworthy on its own.
499         """
500         if len(videos) < 1:
501             raise ExperimentError("compare needs at least one labeled video")
502         eval_a = evaluate_variant(videos, variant_a, iou, honest)
503         eval_b = evaluate_variant(videos, variant_b, iou, honest)
504
505         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
506         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
507
508         by_video_a = {p["video"]: p for p in eval_a["per_video"]}

```

```

509     per_video_delta = [
510         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
511         for p in eval_b["per_video"]]
512     ]
513
514     # One fingerprint over the whole label set ? detects "the deltas were measured
against
515     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
516     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
517
518     result: Dict[str, Any] = {
519         "iou": iou,
520         "label_set_hash": gt_hash(all_gts),
521         "num_videos": len(videos),
522         "variant_a": eval_a,
523         "variant_b": eval_b,
524         "delta": delta,
525         "delta_pct": delta_pct,
526         "per_video_delta": per_video_delta,
527     }
528     if honest_stats:
529         result["honest"] = honest_summary(videos, eval_a, eval_b)
530     return result
531
532
533     # ----- SxS on unlabeled video
534
535
536     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
537         agree_iou: float = 0.5) -> Dict[str, Any]:
538         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
539
540         With no labels there is no lift to measure ? instead this surfaces where the two
541         variants **agree vs diverge**, which is exactly what a human reviews (and what two
542         highlight reels would show side by side):
543
544         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
545         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
546         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
547           the human / a later golden label adjudicates).
548
549         Both variants must be predictable without labels: a learned variant therefore needs
a
550         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
551         ``metrics.match_intervals`` ? no new matching logic.
552         """
553         preds_a = predict(variant_a, video)
554         preds_b = predict(variant_b, video)
555         mr = match_intervals(preds_a, preds_b, agree_iou)
556
557         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
558             for m in mr.matches]
559         agree_iou = [m.iou for m in mr.matches]
560         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
561         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
562
563         def _dur(ivs: List[Interval]) -> float:
564             return sum(e - s for s, e in ivs)
565
566         return {
567             "video": video.name,
568             "agree_iou": agree_iou,
569             "variant_a": variant_a.name,
570             "variant_b": variant_b.name,

```

```

571         "num_a": len(preds_a),
572         "num_b": len(preds_b),
573         "num_agreed": len(agreed),
574         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
575         "duration_a": _dur(preds_a),
576         "duration_b": _dur(preds_b),
577         "agreed": agreed,
578         "a_only": a_only,
579         "b_only": b_only,
580     }
581
582
583     # ----- leaderboard / DB
584
585
586     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
587                          timestamp: Optional[str] = None) -> Dict[str, Any]:
588         """Append a compact, reproducible record of a `compare()` result to the JSON
589         leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
590         `regression_baseline.json`; deliberately the size of a baseline file, not a service
591         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
592         """
593         row: Dict[str, Any] = {
594             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
595             "iou": result.get("iou"),
596             "label_set_hash": result.get("label_set_hash"),
597             "num_videos": result.get("num_videos"),
598             "variant_a": {"name": result["variant_a"]["variant"],
599                          "spec_hash": result["variant_a"]["spec_hash"],
600                          "aggregate": result["variant_a"]["aggregate"]},
601             "variant_b": {"name": result["variant_b"]["variant"],
602                          "spec_hash": result["variant_b"]["spec_hash"],
603                          "aggregate": result["variant_b"]["aggregate"]},
604             "delta": result.get("delta"),
605         }
606         # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
607         the
608         # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
609         honest = result.get("honest") or {}
610         if honest.get("paired_delta_f1"):
611             row["honest_delta_f1"] = honest["paired_delta_f1"]
612         entries: List[Dict[str, Any]] = []
613         if os.path.isfile(path):
614             with open(path, encoding="utf-8") as f:
615                 entries = json.load(f)
616         entries.append(row)
617         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
618         with open(path, "w", encoding="utf-8") as f:
619             json.dump(entries, f, indent=2)
620         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
621                    row["variant_a"]["name"], row["variant_b"]["name"],
622                    (result.get("delta") or {}).get("f1", 0.0), path)
623         return row
624
625     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
626                             gts: Optional[List[Interval]] = None,
627                             feature_names: Sequence[str] = ()) -> VideoCandidates:
628         """Bridge persisted detection ? `VideoCandidates` via
629         `Database.query_candidate_segments`.
630         ``feature_names`` selects which persisted features to lift out of each row's
631         ``stats``
632         JSON (``duration`` is derived from the window bounds). This is the only DB-
633         touching

```

```

632         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
633         but works against any `Database` (including an in-memory one in tests).
634         """
635         rows = db.query_candidate_segments(video_id, detector=detector)
636         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
637         features: Dict[str, List[float]] = {}
638         for name in feature_names:
639             if name == "duration":
640                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
641             else:
642                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
643         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
644

```

## 9. assistant (2026-06-22T16:12:26.323Z)

Now let me read the promotion module and verify the consent guard in context:

## 10. user (2026-06-22T16:12:27.463Z)

```

1         """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2         at the SERVED windowing.
3
4         **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5         honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6         it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7         PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8         cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9         regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10        served windowing** is.
11
12        This module composes ? it adds no new statistics. It layers ONE rule on top of
13        ``should_promote_for_user``:
14
15        A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16        REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17        estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19        A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20        can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21        verdicts so the eval?serve contrast is explicit and auditable.
22
23        Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24        paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25        """
26        from __future__ import annotations
27
28        from dataclasses import dataclass, field

```

```

29     from typing import Any, Dict, Optional, Sequence
30
31     from backend.eval.eval_stats import paired_delta_ci
32     from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34     __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37     def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                          eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39         """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41         ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42         (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43         a single ``regresses`` boolean:
44
45         - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46         the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47         - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48         ``-eps`` and the CI is not a confident net-negative).
49
50         ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51         positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52         """
53         delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54         mean_delta = float(delta["mean_delta"])
55         ci_low = float(delta["ci_low"])
56         ci_high = float(delta["ci_high"])
57         # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58         mean_regresses = mean_delta < -eps
59         ci_confidently_negative = ci_high < -eps
60         regresses = bool(mean_regresses or ci_confidently_negative)
61         return {
62             "regresses": regresses,
63             "mean_delta": mean_delta,
64             "ci_low": ci_low,
65             "ci_high": ci_high,
66             "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67             "sign_test": delta["sign_test"],
68             "n": int(delta["n"]),
69             "eps": float(eps),
70         }
71
72
73     @dataclass
74     class ServedGateResult:
75         """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76         proposal-windowing view, so the eval?serve contrast is never hidden.
77
78         - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79         *and* the served check did not veto (the cue does not regress at the served
windowing).
80         - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not

```

```

regress served).
81     - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82         served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83     - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84         *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85     - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86         tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87     - ``reason`` ? short human-readable explanation of the final verdict.
88     """
89
90     promote: bool
91     served_safe: bool
92     vetoed_by_served: bool
93     reason: str
94     base_decision: PromotionDecision
95     served: Dict[str, Any]
96     proposal: Optional[Dict[str, Any]] = field(default=None)
97
98     def as_dict(self) -> Dict[str, Any]:
99         return {
100             "promote": self.promote,
101             "served_safe": self.served_safe,
102             "vetoed_by_served": self.vetoed_by_served,
103             "reason": self.reason,
104             "base_decision": self.base_decision.as_dict(),
105             "served": self.served,
106             "proposal": self.proposal,
107         }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],
112         served_b: Sequence[float],
113         *,
114         proposal_a: Optional[Sequence[float]] = None,
115         proposal_b: Optional[Sequence[float]] = None,
116         structural: bool = False,
117         min_n: int = 5,
118         p_threshold: float = 0.05,
119         served_regress_eps: float = 0.0,
120         n_boot: int = 2000,
121         seed: int = 0,
122     ) -> ServedGateResult:
123         """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
124
125         The honest bar (``should_promote_for_user``: min-N floor + ?F1 CI excludes 0 + sign
test) is
126         applied to the **served** scores ? the operating point production runs ? so a
proposal-only
127         win can never carry a promotion. The optional ``proposal_`` scores are scored too
and
128         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
129         own).
130
131         Args:
132             served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133             measured at the **SERVED** windowing. These drive the decision.
134             proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).

```

```

Reported for
135         contrast; the served regression guard still has the final say.
136         structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
137         held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138         the served default if it regresses served. So for ``structural=True`` we report
139         ``keep_on`` honestly but ``promote`` reflects the served-safety check.
140         served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143         """
144         # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
145         base = should_promote_for_user(
146             served_a, served_b,
147             structural=structural, min_n=min_n, p_threshold=p_threshold,
148             n_boot=n_boot, seed=seed,
149         )
150
151         # 2) The eval?serve guard: does B regress A at the served windowing?
152         served = served_regresses(served_a, served_b, eps=served_regress_eps,
153                                   n_boot=n_boot, seed=seed)
154         served_safe = not served["regresses"]
155
156         # 3) Optional proposal-windowing view (reported, never promoting on its own).
157         proposal: Optional[Dict[str, Any]] = None
158         if proposal_a is not None and proposal_b is not None:
159             proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
160                                         n_boot=n_boot, seed=seed)
161
162         # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
163         if structural:
164             # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
165             # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
166             # served, promote=False (don't bake an eval-only win into production) while
keep_on holds.
167             promote = served_safe
168             vetoed = False
169             if served_safe:
170                 reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                           f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172             else:
173                 reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                           f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                           "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                           f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f}")
183                 f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "

```

```

184                 "windowing production doesn't serve cannot promote.")
185         elif would_promote:
186             promote = True
187             vetoed = False
188             reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                     f"{served['mean_delta']:+.4f}). " + base.reason)
190         else:
191             promote = False
192             vetoed = False
193             reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204

```

#### 11. user (2026-06-22T16:12:29.510Z)

Found 4 files  
docs\COMPUTE\_DECOUPLED\_SERVING\README.md  
tests\test\_flywheel.py  
backend\flywheel.py  
backend\api\job\_worker.py

#### 12. assistant (2026-06-22T16:12:31.754Z)

Now let me verify capture\_job is only called through run\_for\_job and check for any logging that could leak PII:

#### 13. user (2026-06-22T16:12:33.830Z)

```

54: def capture_job(
63:     (``video_id`` + rally output + provenance); when a local ``source_path`` is given, also
stores the
71:         json.dump({"video_id": workspace.video_id, "rally_output": rally_output}, fh,
indent=2)
138:         logger.info("[FLYWHEEL] job %s not captured ? no attested consent (faked-deny).",
job.get("id"))
141:     video_id = (result or {}).get("video_id")
142:     if not video_id:
143:         logger.info("[FLYWHEEL] job %s has no video_id ? nothing to capture.",
job.get("id"))
153:     ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
154:     written = capture_job(
160:     logger.info("[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id,
written)
161:     return {"captured": written, "video_id": video_id}

```

#### 14. user (2026-06-22T16:12:34.813Z)

```

72: def _run_claimed_job(job: Dict[str, Any]) -> None:
115:     claimed row (carries ``owner_id``); ``result`` is the pipeline output.""
126:         owner_id=job.get("owner_id"), margin=billing_cfg.margin,
141:     row (carries ``owner_id``); ``result`` is the pipeline output.""
150:     flywheel.run_for_job(job, result, _main.global_config)
167:     job = _main.db_instance.claim_due_job()
171:     _main._run_claimed_job(job)

```

### 15. assistant (2026-06-22T16:12:36.702Z)

Let me check if there's any path where the source\_path parameter could be passed without proper guards:

### 16. user (2026-06-22T16:12:37.781Z)

```
backend/api/job_worker.py:150:         flywheel.run_for_job(job, result, _main.global_config)
backend/flywheel.py:122:def run_for_job(
```

### 17. assistant (2026-06-22T16:12:39.676Z)

Now let me check the full context of the job\_worker hook to see if source\_path could be passed:

### 18. user (2026-06-22T16:12:40.575Z)

```
130         logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)
131
132
133     def _maybe_run_flywheel(job: Dict[str, Any], result: Dict[str, Any]) -> None:
134         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
135         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
136         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
137         land), so this captures **nothing** in production. When activated it captures the
consented job's
138         artifacts into the per-video workspace (? later labeling ? the golden training set).
139
140         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
141         row (carries ``owner_id``); ``result`` is the pipeline output."""
142         import backend.main as _main
143
144         flywheel_cfg = getattr(_main.global_config, "flywheel", None)
145         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
146             return
147         try:
148             from backend import flywheel
149
150             flywheel.run_for_job(job, result, _main.global_config)
151         except Exception as e: # never wedge a completed job on the flywheel
152             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
153
154
155     def _drain_due_jobs() -> int:
156         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
157         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
158
159         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
160         up by a FUTURE drain once due; we schedule that follow-up drain via
`schedule_drain` so
161         a single-worker box keeps making progress with no central timer (the DB is the
clock).
162         Returns the number of jobs that reached a terminal/parked state this tick.
163         """
164         import backend.main as _main
165         ran = 0
166         while True:
167             job = _main.db_instance.claim_due_job()
```

```
168         if job is None:
169             break
```

## 19. assistant (2026-06-22T16:12:42.847Z)

Good! The hook doesn't pass `source_path`. Now let me verify the `consent_attested` behavior more carefully and check for any code paths that could bypass it:

## 20. user (2026-06-22T16:12:43.732Z)

```
122     def run_for_job(
123         job: Dict[str, Any],
124         result: Dict[str, Any],
125         config: Any,
126         *,
127         source_path: Optional[str] = None,
128     ) -> Optional[Dict[str, Any]]:
129         """The gated orchestrator called at job completion. Returns ``None`` (a no-op)
130         unless both
131         ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it never
132         captures (consent is
133         faked-deny). On a consented job it captures the artifacts into the per-video
134         workspace; shadow-eval
135         + promotion are wired but stay inert until the owned model + golden labels exist.
136         Best-effort: the
137         caller wraps this so a bookkeeping hiccup never wedges a completed job."""
138         fw = getattr(config, "flywheel", None)
139         if not fw or not getattr(fw, "enabled", False):
140             return None
141         if not consent_attested(job, config):
142             logger.info("[FLYWHEEL] job %s not captured ? no attested consent (faked-
143             deny).", job.get("id"))
144             return None
145         video_id = (result or {}).get("video_id")
146         if not video_id:
147             logger.info("[FLYWHEEL] job %s has no video_id ? nothing to capture.",
148             job.get("id"))
149             return None
150         storage_cfg = getattr(config, "storage", None)
151         output_dir = getattr(getattr(config, "output", None), "output_dir", "output")
152         root = _workspace.resolve_workspace_root(
153             getattr(storage_cfg, "workspace_root", "") or "",
154             getattr(storage_cfg, "output_root", "") or "",
155             output_dir,
156         )
157         ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
158         written = capture_job(
159             ws,
160             rally_output={"play_segments": (result or {}).get("play_segments", [])},
161             source_path=source_path,
162             storage_config=storage_cfg,
163         )
164         logger.info("[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id,
165         written)
166         return {"captured": written, "video_id": video_id}
```

## 21. assistant (2026-06-22T16:12:49.879Z)

Perfect! Now let me review the key facts from my analysis:

1. **Two independent guards**: Lines 135-139 in `flywheel.py` check BOTH `flywheel.enabled` AND `consent_attested()`

2. **\*\*Consent is truly a hard-deny\*\*** (lines 43-51): returns `False` always, documented with "Do not relax without counsel sign-off"
3. **\*\*capture\_job is ONLY called via run\_for\_job\*\*** - already confirmed (line 150 in job\_worker.py is the only call)
4. **\*\*Source\_path is NOT passed from the hook\*\*** - the job\_worker hook at line 150 calls `flywheel.run\_for\_job(job, result, \_main.global\_config)` with no source\_path parameter, so source\_path defaults to None
5. **\*\*Logging only uses job.get("id") and video\_id\*\*** - no owner\_id leakage in the new logging (lines 138, 143, 160)
6. **\*\*Video\_id is derived from result\*\*** - result comes from the pipeline output, not user input

Let me now check if there's any path where the config could be tampered with and verify default values:

## 22. user (2026-06-22T16:12:50.769Z)

```
384         margin: float = 0.0                                # resale markup (?0); 0 = bill at
cost
385
386
387     class FlywheelConfig(BaseModel):
388         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
389         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
390         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
391         the owned model climbs toward the D11 ship floor.
392
393         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
394         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
395         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
396         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
397         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
398         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
399         owner/counsel/GPU scope. Pairs with M9-consent."""
400
401         enabled: bool = False
402         workspace_prefix: str = "flywheel"    # per-video workspace prefix for captured
artifacts
403         shadow_eval: bool = True              # run the owned-vs-Gemini shadow comparison on
capture
404         min_promote_n: int = 5                # promote_if_served_safe `min_n` (the served-
safe gate)
405
406
407     class AppConfig(BaseModel):
408         """Root configuration object.
409
410         All runtime configuration should be accessed via an instance of this class
411         (or the loader) rather than raw dict lookups.
412         """
413
414         sport: Sport = "badminton"
415         ai_provider: str = "gemini"
416         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
417         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
```

```

418     ai_model: str = "gemini-flash-latest"
419
420     validation: ValidationConfig = Field(default_factory=ValidationConfig)
421     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
422     output: OutputConfig = Field(default_factory=OutputConfig)
423     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
424     security: SecurityConfig = Field(default_factory=SecurityConfig)
425     logging: LoggingConfig = Field(default_factory=LoggingConfig)
426     storage: StorageConfig = Field(default_factory=StorageConfig)
427     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
428     billing: BillingConfig = Field(default_factory=BillingConfig)
429     flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
430
431     # Allow extra keys during transition so existing config.json files do not break.
432     model_config = {
433         "extra": "allow",

```

### 23. assistant (2026-06-22T16:12:53.995Z)

Excellent! Now let me check the test file once more to verify the consent override is only used in tests:

### 24. user (2026-06-22T16:12:55.207Z)

```

53     def test_run_for_job_captures_when_enabled_and_consented(tmp_path, monkeypatch):
54         """With consent monkeypatched True, a per-video manifest lands under
<output>/flywheel/<vid>/."""
55         monkeypatch.setattr(flywheel, "consent_attested", lambda job, config=None: True)
56         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
57         out = flywheel.run_for_job(
58             {"id": 7}, {"video_id": "abc123", "play_segments": [{"id": 1, "start_time":
0}}], cfg)
59         assert out and out["video_id"] == "abc123"
60         manifest = tmp_path / "flywheel" / "abc123" / "outputs" / "capture.json"
61         assert manifest.exists()
62         data = json.loads(manifest.read_text())
63         assert data["video_id"] == "abc123"
64         assert data["rally_output"]["play_segments"][0]["id"] == 1
65
66
67     def test_run_for_job_no_video_id_is_noop(tmp_path, monkeypatch):

```

### 25. assistant (2026-06-22T16:12:57.454Z)

Perfect - the consent override is test-only via monkeypatch. Now let me verify the capture\_job source video logic more carefully:

### 26. user (2026-06-22T16:12:58.334Z)

```

54     def capture_job(
55         workspace: "_workspace.VideoWorkspace",
56         *,
57         rally_output: Dict[str, Any],
58         source_path: Optional[str] = None,
59         storage_config: Any = None,
60     ) -> Dict[str, str]:
61         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
62         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
63         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
64         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``

```

```

65         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""
66         written: Dict[str, str] = {}
67         cfg = _sc(storage_config)
68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
fh, indent=2)
72                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
73                 written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
74                 if source_path and os.path.exists(source_path):
75                     source_uri = _workspace.join_uri(workspace.source,
os.path.basename(source_path))
76                     written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
77         finally:
78             try:
79                 os.unlink(manifest_local)
80             except OSError:
81                 pass
82         return written
83

```

## 27. assistant (2026-06-22T16:13:00.606Z)

Now let me verify this matches what M8 (the cost hook) does for consistency:

## 28. user (2026-06-22T16:13:01.568Z)

```

115         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
116         import backend.main as _main
117
118         billing_cfg = getattr(_main.global_config, "billing", None)
119         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
120             return
121         try:
122             usage = (result or {}).get("usage") or {}
123             _main.db_instance.record_job_cost(
124                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
125                 gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
126                 owner_id=job.get("owner_id"), margin=billing_cfg.margin,
127             )
128             logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
129         except Exception as e: # never wedge a completed job on bookkeeping
130             logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)

```

## 29. assistant (2026-06-22T16:13:04.041Z)

Perfect! The M8 hook mirrors the M11 structure: `enabled` check first, then try-except best-effort. Now let me verify one final concern - that the manifest doesn't accidentally capture owner\_id:

## 30. user (2026-06-22T16:13:04.919Z)

```

68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
fh, indent=2)
72                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")

```

### 31. assistant (2026-06-22T16:13:10.196Z)

Excellent! The manifest at line 71 only contains `video\_id` and `rally\_output`, nothing that could leak owner PII. Now let me verify consent\_attested returns the correct type and make one final check on the guard logic:

### 32. user (2026-06-22T16:13:10.898Z)

```
43     def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
44         """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).
45
46         The flywheel may capture a user's footage into the training pool ONLY with attested,
**adults-only**
47         consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
separate explicit
48         opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent
columns + an
49         adults-only attestation + a retention class ? lands with **M9-consent**; until then
this denies
50         every job so the harness captures nothing. **Do not relax without counsel sign-
off.**"""
51         return False
52
```

### 33. assistant (2026-06-22T16:13:13.782Z)

Perfect! Now let me verify one last crucial point - that there's absolutely NO way to reach capture\_job without going through BOTH guards in run\_for\_job:

### 34. user (2026-06-22T16:13:14.962Z)

```
backend/config/models.py:403:     shadow_eval: bool = True                # run the owned-vs-Gemini
shadow comparison on capture
backend/flywheel.py:54: def capture_job(
backend/flywheel.py:85: def shadow_eval(
backend/flywheel.py:103:         raise ValueError("unlabeled shadow_eval compares exactly one
video at a time")
backend/flywheel.py:107: def propose_promotion(
backend/flywheel.py:154:     written = capture_job(
```

### 35. assistant (2026-06-22T16:13:17.475Z)

Perfect! `capture\_job` is ONLY called at line 154 in `flywheel.py`, which is inside `run\_for\_job`, after BOTH guards have passed. Let me do one final comprehensive verification of the guard logic flow:

### 36. user (2026-06-22T16:13:18.415Z)

```
122     def run_for_job(
123         job: Dict[str, Any],
124         result: Dict[str, Any],
125         config: Any,
126         *,
127         source_path: Optional[str] = None,
128     ) -> Optional[Dict[str, Any]]:
129         """The gated orchestrator called at job completion. Returns ``None`` (a no-op)
unless **both**
130         ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it never
captures (consent is
131         faked-deny). On a consented job it captures the artifacts into the per-video
workspace; shadow-eval
132         + promotion are wired but stay inert until the owned model + golden labels exist.
Best-effort: the
133         caller wraps this so a bookkeeping hiccup never wedges a completed job."""
```

```

134         fw = getattr(config, "flywheel", None)
135         if not fw or not getattr(fw, "enabled", False):
136             return None
137         if not consent_attested(job, config):
138             logger.info("[FLYWHEEL] job %s not captured ? no attested consent (faked-
deny).", job.get("id"))
139             return None
140
141         video_id = (result or {}).get("video_id")
142         if not video_id:
143             logger.info("[FLYWHEEL] job %s has no video_id ? nothing to capture.",
job.get("id"))
144             return None
145
146         storage_cfg = getattr(config, "storage", None)
147         output_dir = getattr(getattr(config, "output", None), "output_dir", "output")
148         root = _workspace.resolve_workspace_root(
149             getattr(storage_cfg, "workspace_root", "") or "",
150             getattr(storage_cfg, "output_root", "") or "",
151             output_dir,
152         )
153         ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
154         written = capture_job(
155             ws,
156             rally_output={"play_segments": (result or {}).get("play_segments", [])},
157             source_path=source_path,
158             storage_config=storage_cfg,
159         )
160         logger.info("[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id,
written)
161         return {"captured": written, "video_id": video_id}

```

### 37. assistant (2026-06-22T16:13:30.652Z)

Perfect! Now I have all the information I need. Let me compile my comprehensive findings:

## Summary of Analysis

I've performed an adversarial privacy/inertness review of the M11 flywheel skeleton. Here are my key findings:

#### \*\*GUARD FLOW:\*\*

1. **\*\*Line 135\*\***: ``if not fw or not getattr(fw, "enabled", False): return None`` ? First guard: config must have ``flywheel.enabled=True`` (defaults False)
2. **\*\*Line 137-139\*\***: ``if not consent_attested(job, config): ... return None`` ? Second guard: consent check (currently always returns False)
3. **\*\*Line 154\*\***: ``capture_job()`` is ONLY reachable after BOTH guards pass

#### \*\*EVIDENCE OF INERTNESS:\*\*

- ``FlywheelConfig.enabled`` defaults to ``False`` (backend/config/models.py:401)
- ``consent_attested()`` returns ``False`` always (backend/flywheel.py:51) ? documented as "FAKED hard-deny" with "Do not relax without counsel sign-off"
- ``capture_job()`` has no other callers ? only path is line 154 in ``run_for_job``
- ``_maybe_run_flywheel()`` in `job_worker.py` calls ``run_for_job(job, result, config)`` with NO `source_path` parameter (line 150), so `source_path` defaults to None
- Manifest JSON only contains ``video_id`` + ``rally_output`` (line 71), no `owner_id` or user PII
- Hook is wrapped in best-effort try-except (lines 147-152) ? exceptions don't propagate

#### \*\*NO LOGGING LEAKS:\*\*

- Logging only uses ``job.get("id")`` and ``video_id`` derived from pipeline result
- No `owner_id` logged anywhere in the new code

#### \*\*SCHEMA SAFETY:\*\*

- No DB migration in this change ? consent schema is deferred

- All guards are in code, not database dependent

#### **\*\*VERIFICATION VIA TESTS:\*\***

- Test confirms disabled-by-default is a no-op (test\_run\_for\_job\_noop\_when\_disabled)
- Test confirms consent-denied is a no-op even when enabled (test\_run\_for\_job\_noop\_when\_enabled\_but\_no\_consent)
- Test confirms enabled+consented writes to workspace (test\_run\_for\_job\_captures\_when\_enabled\_and\_consented uses monkeypatch only)

#### **\*\*CONTROL/PROPOSAL ARGUMENT ORDER:\*\***

Verified in flywheel.py line 119: `promote\_if\_served\_safe(served\_gemini, served\_owned, ...)` ? Gemini is control (A), owned is proposal (B), matching promotion.py line 145-149's expectation.

### **38. user (2026-06-22T16:13:49.944Z)**

Structured output provided successfully

### **39. assistant (2026-06-22T16:13:59.443Z)**

Perfect! My review is complete. The M11 flywheel skeleton is **safe to ship**. Here's the executive summary:

---

## **M11 Flywheel Privacy/Inertness Review: ? SAFE**

**Bottom line:** The change ships a **default-OFF, privacy-sensitive data capture harness** with **two independent, watertight guardrails** that make it impossible to capture any user data in production today:

#### **The Double Lock:**

1. **flywheel.enabled=False** (default) ? the hook is a strict no-op unless explicitly enabled in config
2. **consent\_attested()** returns **False** always (faked hard-deny) ? even if enabled, the function denies every job with transparent, counsel-reviewable code

#### **Path to Capture:**

Data can **ONLY** flow through:

- **maybe\_run\_flywheel()** hook (job\_worker.py:150) ? checks enabled ?
- **run\_for\_job()** (flywheel.py:122) ? checks enabled at line 135, checks consent at line 137, BOTH must pass
- Only then does **capture\_job()** execute (line 154)

#### **What Gets Captured (if both guards pass):**

- A small **capture.json** manifest with only **video\_id** + **rally\_output** (no owner/user PII)
- Source video: NOT captured by hook (no source\_path passed from job\_worker:150)

#### **Logging:**

- Only logs **job.id** + **video\_id**, no PII leak

#### **Test Coverage:**

- ? Disabled-by-default is a no-op
- ? Enabled-but-no-consent is a no-op
- ? Enabled+consented (via monkeypatch in tests only) writes manifest correctly
- ? Hook is best-effort (exceptions don't wedge jobs)

**Inert guarantee: YES** ? it is truly impossible for the system to capture any user data in production today.